

System and Device Programming Projects

Device Programming Part (Prof. Quer)

Project Q1

Graph Neural Network Inference on GPUs

Project's summary

Graph Neural Networks (GNNs) extend deep learning models to graph-structured data, with applications in recommender systems, social network analysis, cybersecurity, bioinformatics, and industrial recommendation.

The goal of this project is to develop an inference engine for GNNs on multi-core CPUs (C++) and GPUs (CUDA), and to evaluate how different parallelization strategies (vertex parallel, edge parallel, message-passing batching, sparse versus dense representations) affect performance and scalability on large graphs.

Problem definition

Consider a directed or undirected graph $G = (V, E)$ with node features and, optionally, edge features. A typical GNN layer performs, for each node $v \in V$:

1. Aggregation of neighbors' features (sum, mean, or attention-based weighted combination).
2. Update of the node representation through a neural function (e.g., MLP).

The project requires:

- Implement one or two basic GNN architectures (e.g., GCN and/or GraphSAGE) for inference only, assuming pre-loaded weights.
- Design a sequential C/C++ implementation operating on efficient graph formats (CSR/CSC).
- Design a parallel CPU implementation (explicit threads, tasks, or OpenMP) exploiting parallelism over nodes, edges, or layers.
- Design one or more CUDA versions exploring different mappings (thread per node, per edge, per feature; use of shared memory; coalesced access on CSR structures).

Compare performance in terms of:

- Throughput (nodes/s or edges/s), memory footprint, and scalability with respect to the graph size (from tens of thousands up to millions of nodes).
- Feature vector size.
- Model depth (number of layers).

Implementation Details

Assume a single, large, static graph $G = (V, E)$, directed or undirected.

- Each vertex $v \in V$ has:
 - A unique integer id $v \in \{0, \dots, |V| - 1\}$.
 - A feature vector $x_v \in \mathbb{R}^F$ (node attributes).
 - Optionally, a label y_v (e.g., class id) used only for accuracy evaluation on a node-classification task.
- Each edge $(u, v) \in E$ has:

- **Optional** edge feature vector $e_{uv} \in \mathbb{R}^{F_e}$ (for GraphSAGE with edge features or attention mechanisms).

A GNN layer performs, for every node v :

1. Aggregation

- Collect neighbor representations $h_u^{(l)}$ for $u \in \mathcal{N}(v)$.
- Apply an aggregation operator, e.g.:
 - Sum: $m_v^{(l)} = \sum_{u \in \mathcal{N}(v)} h_u^{(l)}$.
 - Mean: $m_v^{(l)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} h_u^{(l)}$.
 - Attention-weighted sum (optional extension).

2. Update

- Combine current node state and aggregated message through a fixed neural function:
 - GCN-style: $h_v^{(l+1)} = \sigma(W^{(l)} m_v^{(l)})$
 - GraphSAGE-style: $h_v^{(l+1)} = \sigma(W^{(l)} \cdot \text{concat}(h_v^{(l)}, m_v^{(l)}))$
- Weights $W^{(l)}$ and all layer parameters are preloaded and fixed (inference only).

The overall model is a stack of L such layers:

- Input: $h_v^{(0)} = x_v$.
- For $l = 0, \dots, L - 1$, compute $h_v^{(l+1)}$ for all v by applying the layer in parallel on the graph.
- Output:
 - Node classification: apply a final linear layer or softmax on $h_v^{(L)}$ to produce logits for each node, then (optionally) compute accuracy on a given split.
 - You can explicitly state "focus on full-batch inference on a single large graph, not mini-batches across many small graphs." [file:1]

The parallelization dimensions to explore are across nodes, edges, feature dimensions, and possibly across layers. Students should treat each GNN layer as one "graph-wide iteration" of message passing. For performance evaluation, each configuration (graph, feature size, depth, implementation) should be run multiple times to get stable timings.

Public datasets and formats

Large graphs with node features for GNNs are publicly available:

- Open Graph Benchmark (OGB) - large, realistic graphs for node and link prediction, with standardized loaders and splits.
 - Website: <https://ogb.stanford.edu>
 - Datasets such as ogbn-products, ogbn-papers100M, ogbn-arxiv provide:
 - A single large graph.
 - Node features and labels.
 - Standard tasks and splits.
 - The official PyTorch Geometric / DGL loaders expose the graph in CSR-like adjacency structures; students can export adjacency and features to your own CSR + dense-matrix format.
- Planetoid / citation datasets (Cora, CiteSeer, PubMed) - classic small-to-medium node-classification benchmarks with node features and labels.
 - Hosted in many repos; a canonical reference is the Planetoid data used by GCN and PyG.

- Even if distributed in preprocessed NumPy/SciPy form, they are fundamentally sparse adjacency matrices + dense node feature matrices, directly mappable to CSR + dense features.
 - They can serve as “small graphs” for functional debugging and as part of the required “public benchmarks” in your text.
- For synthetic graphs, students can generate:
 - Scale-free (Barabási-Albert), Erdos-Rényi random, and small-world graphs using standard libraries, then export to CSR.

Required background

The required background is covered by:

- Advanced programming courses (data structures, graphs, memory management).
- “System and Device Programming” course (concurrent programming, memory models).
- Basic CUDA programming (thread hierarchy, memory hierarchy, synchronization).

A short introduction to GNNs can be provided via selected survey/tutorial papers; students are not required to implement training, only the forward pass.

Working Environment

Students may work on a laptop or desktop, under Unix-like or Windows systems.

The implementation must be done in C/C++ with CUDA extensions for the GPU part.

The use of the following is encouraged:

- Lightweight utilities to load graph files (edge lists or custom CSR).
- Profiling tools (nvprof, Nsight Systems/Compute, perf) for performance analysis.

The project can naturally be extended into an MSc thesis by adding mini-batching across multiple graphs, graph partitioning and load-balancing techniques, or multi-GPU support.

Deliverables

All projects must be delivered including:

- C/C++ and CUDA source files.
- A README text file (plain ASCII) with instructions on how to compile and run the program, execution parameters, and dataset formats.
- A DOCUMENTATION file (Word, LaTeX, Markdown, etc.) containing:
 - main design choices (graph format, memory layout, CPU and GPU parallelization schemes, strategies for handling highly skewed degree distributions);
 - an experimental evaluation with tables and plots comparing:
- Sequential vs multi-core vs GPU implementations;
 - different parallelization strategies (vertex-centric vs edge-centric, with/without shared memory, with/without feature compression);
 - different graph and feature sizes.
 - experiments on both synthetic graphs (scale-free, random, small-world generators) and public graph benchmarks cited in the references.
- A set of OVERHEAD slides (PowerPoint or similar) for a 20-minute oral presentation of the projects.

Contact

Prof. Stefano Quer (stefano.quer@polito.it).

References

- Course material for "System and Device Programming", for general parallel programming concepts.
- Introductory tutorials and surveys on Graph Neural Networks and message-passing architectures.
- NVIDIA CUDA documentation (Programming Guide, Best Practices) for memory and parallelism optimizations on GPUs.
- Research papers on efficient GPU implementations of GNNs and on optimized sparse matrix operations.